CSA0619-Design and Analysis of Algorithm

CO4_AT1- Code Challenge

Q28 – Weighted Interval Scheduling

Code:

main.py

```python
def weighted_interval_scheduling(intervals):
    intervals.sort(key=lambda x: x[1])

    n = len(intervals)
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        start, end, weight = intervals[i - 1]
        j = i - 2
        while j >= 0 and intervals[j][1] > start:
            j -= 1
        include = weight + dp[j + 1]
        exclude = dp[i - 1]
        dp[i] = max(include, exclude)
    return dp[n]
n = int(input("Enter number of intervals: "))
intervals = []
for i in range(n):
    start, end, weight = map(
        int,
        input(f"Enter start, end and weight for interval {i + 1}: ").split()
    )
    intervals.append((start, end, weight))

# Calculate maximum weight
max_weight = weighted_interval_scheduling(intervals)

print("Max Weight =", max_weight)
```

Output:

```
Enter number of intervals: 3
Enter start, end and weight for interval 1: 1 3 5
Enter start, end and weight for interval 2: 2 5 6
Enter start, end and weight for interval 3: 4 6 5
Max Weight = 10


...Program finished with exit code 0
Press ENTER to exit console.
```

Dynamic Programming Analysis:

Let dp[i] represent the maximum weight obtainable using the first i intervals.
The recurrence is:
$dp[i]=max(dp[i-1], weighti+dp[j])$

Time Complexity:

$O(n2)$

Space Complexity:

$O(n)$